



PALADIN
BLOCKCHAIN SECURITY

Smart Contract Security Assessment

Preliminary Report

For LayerZero (Euler)

23 January 2025



paladinsec.co



info@paladinsec.co

Table of Contents

Table of Contents	2
Disclaimer	3
1 Overview	4
1.1 Summary	4
1.2 Contracts Assessed	4
1.3 Findings Summary	5
1.3.1 MintBurnOFTAdapter	6
1.3.2 OFTAdapterUpgradeable	6
1.3.3 ERC20BurnableMintable	6
2 Findings	7
2.1 MintBurnOFTAdapter	7
2.1.1 Privileged Functions	7
2.1.2 Issues & Recommendations	8
2.2 OFTAdapterUpgradeable	9
2.2.1 Privileged Functions	9
2.2.2 Issues & Recommendations	10
2.3 ERC20BurnableMintable	11
2.3.1 Privileged Functions	11
2.3.2 Issues & Recommendations	11



Disclaimer

Paladin Blockchain Security ("Paladin") has conducted an independent audit to verify the integrity of and highlight any vulnerabilities or errors, intentional or unintentional, that may be present in the codes that were provided for the scope of this audit. This audit report does not constitute agreement, acceptance or advocacy for the Project that was audited, and users relying on this audit report should not consider this as having any merit for financial advice in any shape, form or nature. The contracts audited do not account for any economic developments that may be pursued by the Project in question, and that the veracity of the findings thus presented in this report relate solely to the proficiency, competence, aptitude and discretion of our independent auditors, who make no guarantees nor assurance that the contracts are completely free of exploits, bugs, vulnerabilities or deprecation of technologies. Further, this audit report shall not be disclosed nor transmitted to any persons or parties on any objective, goal or justification without due written assent, acquiescence or approval by Paladin.

All information provided in this report does not constitute financial or investment advice, nor should it be used to signal that any persons reading this report should invest their funds without sufficient individual due diligence regardless of the findings presented in this report. Information is provided 'as is', and Paladin is under no covenant to the completeness, accuracy or solidity of the contracts audited. In no event will Paladin or its partners, employees, agents or parties related to the provision of this audit report be liable to any parties for, or lack thereof, decisions and/or actions with regards to the information provided in this audit report.

Cryptocurrencies and any technologies by extension directly or indirectly related to cryptocurrencies are highly volatile and speculative by nature. All reasonable due diligence and safeguards may yet be insufficient, and users should exercise considerable caution when participating in any shape or form in this nascent industry.

The audit report has made all reasonable attempts to provide clear and articulate recommendations to the Project team with respect to the rectification, amendment and/or revision of any highlighted issues, vulnerabilities or exploits within the contracts provided. It is the sole responsibility of the Project team to sufficiently test and perform checks, ensuring that the contracts are functioning as intended, specifically that the functions therein contained within said contracts have the desired intended effects, functionalities and outcomes of the Project team.

Paladin retains the right to re-use any and all knowledge and expertise gained during the audit process, including, but not limited to, vulnerabilities, bugs, or new attack vectors. Paladin is therefore allowed and expected to use this knowledge in subsequent audits and to inform any third party, who may or may not be our past or current clients, whose projects have similar vulnerabilities. Paladin is furthermore allowed to claim bug bounties from third-parties while doing so.

1 Overview

This report has been prepared for LayerZero's Mint and Burn non-upgradeable contracts on the Ethereum network. Paladin provides a user-centred examination of the smart contracts to look for vulnerabilities, logic errors or other issues from both an internal and external perspective.

1.1 Summary

Project Name	LayerZero
URL	https://layerzero.network/
Platform	Ethereum
Language	Solidity
Preliminary Contracts	https://github.com/euler-xyz/evk-periphery/tree/70b26235cfa1e36627ccdca59a760411c1aa8288
Resolution #1	https://github.com/euler-xyz/evk-periphery/commit/e3699ffb644bb96bfcff4fb6f0f35d11d23f6503

1.2 Contracts Assessed

Name	Contract	Live Code Match
MintBurnOFTAdapter		
OFTAdapterUpgradeable		
ERC20BurnableMintable		

1.3 Findings Summary

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)
● High	0	-	-	-
● Medium	0	-	-	-
● Low	0	-	-	-
● Informational	2	1	-	1
Total	2	1	0	1

Classification of Issues

Severity	Description
● High	Exploits, vulnerabilities or errors that will certainly or probabilistically lead towards loss of funds, control, or impairment of the contract and its functions. Issues under this classification are recommended to be fixed with utmost urgency.
● Medium	Bugs or issues with that may be subject to exploit, though their impact is somewhat limited. Issues under this classification are recommended to be fixed as soon as possible.
● Low	Effects are minimal in isolation and do not pose a significant danger to the project or its users. Issues under this classification are recommended to be fixed nonetheless.
● Informational	Consistency, syntax or style best practices. Generally pose a negligible level of risk, if any.

1.3.1 MintBurnOFTAdapter

ID	Severity	Summary	Status
01	INFO	Bridged tokens on destination chains that are burnable could become locked in the adapter on mainnet	ACKNOWLEDGED

1.3.2 OFTAdapterUpgradeable

ID	Severity	Summary	Status
02	INFO	OFTAdapterUpgradeable is left uninitialized	✓ RESOLVED

1.3.3 ERC20BurnableMintable

No issues found.

2 Findings

2.1 MintBurnOFTAdapter

MintBurnOFTAdapter is a variant of the standard OFT adapter implemented by LayerZero that uses an existing ERC20's mint and burn mechanisms for cross-chain transfers.

It overwrites the original version by removing the intermediary `minterBurner` contract and instead it is granted the `MINTER_ROLE`, allowing it to directly mint underlying tokens. Additionally, when tokens are bridged, the adapter requires the approval of the sender and calls `burnFrom` instead of calling `burn`.

2.1.1 Privileged Functions

- `setMsgInspector` [OWNER]
- `setEnforcedOptions` [OWNER]
- `setPreCrime` [OWNER]
- `setPeer` [OWNER]
- `setDelegate` [OWNER]
- `renounceOwnership` [OWNER]
- `transferOwnership` [OWNER]



2.1.2 Issues & Recommendations

Issue #01	Bridged tokens on destination chains that are burnable could become locked in the adapter on mainnet
Severity	 INFORMATIONAL
Description	Tokens get bridged by first calling <code>OFTAdapterUpgradeable</code> (deployed on mainnet) and locking an amount of tokens in it. The adapter sends a cross-chain message to the endpoint on the destination chain. Once the message is received on the destination chain the <code>MintBurnOFTAdapter</code> is called to mint the proportional amount of tokens that were locked on mainnet. <code>MintBurnOFTAdapter</code> is a modified <code>OFT</code> adapter that is granted the <code>MINTER_ROLE</code> to an underlying <code>ERC20Burnable</code> token contract. Upon <code>receive()</code>, it mints tokens to an address and when tokens are sent back it requires approval from the caller in order to burn the tokens and unlock them on the source chain. The underlying token contract inherits the <code>ERC20Burnable</code> contract which allows token holders to burn their tokens themselves, by calling <code>burn()</code>. A specific case that stems from this is that an address that has been bridged tokens on a destination chain can burn them directly. As result those tokens can never be bridged back and will remain locked in the adapter contract on mainnet.
Recommendation	Consider if the above scenario is expected and acceptable. In case it is not, then a possible approach is to restrict burning in <code>ERC20BurnableMintable</code> only to the adapter contract.
Resolution	 ACKNOWLEDGED
	The team commented that this behavior is expected and acceptable.

2.2 OFTAdapterUpgradeable

OFTAdapterUpgradeable is a contract that adapts an ERC-20 token to the OFT functionality. It leverages the upgradeable version of the OFTAdapter from LayerZero. It works by transferring in, locking and sending cross-chain messages to a MintBurnOFTAdapter on another chain. Similarly, when bridged tokens are sent back, they get unlocked and sent out to the recipient.

2.2.1 Privileged Functions

- `setMsgInspector [OWNER]`
- `setEnforcedOptions [OWNER]`
- `setPreCrime [OWNER]`
- `setPeer [OWNER]`
- `setDelegate [OWNER]`
- `renounceOwnership [OWNER]`
- `transferOwnership [OWNER]`



2.2.2 Issues & Recommendations

Issue #02	OFTAdapterUpgradeable is left uninitialized
Severity	 INFORMATIONAL
Description	OFTAdapterUpgradeable implements the upgradeable version of the OFT contract by LayerZero. Thus, it implements an <code>initialize</code> function which is used to set the state variables in the proxy. It is always recommended that the initializer of the implementation contract sitting behind a proxy is explicitly disabled. This prevents initialization of the implementation by unwanted actors as a <code>_disableInitializers</code> call is required in any constructor of the main contract. In transparent upgradable proxies this does not pose a risk but it is still considered a best practice.
Recommendation	Consider adding the <code>_disableInitializers</code> call in the constructor of <code>OFTAdapterUpgradeable</code> .
Resolution	 RESOLVED

2.3 ERC20BurnableMintable

ERC20BurnableMintable is the underlying token used by the MintBurnOFTAdapter for bridging tokens. The contract inherits the ERC20Burnable implementation by OpenZeppelin, allowing token holders to burn their tokens themselves or through an approval.

It allows the caller with the MINTER_ROLE to mint new tokens. In case of emergency, the caller with the REVOKE_MINTER_ROLE can revoke the MINTER_ROLE from an address.

2.3.1 Privileged Functions

- `revokeMinterRole[REVOKE_MINTER_ROLE]`
- `mint[MINTER_ROLE]`
- `revokeRole[ADMIN]`
- `grantRole[ADMIN]`

2.3.2 Issues & Recommendations

No issues found.



PALADIN
BLOCKCHAIN SECURITY